Guía definitiva trabajo final CNVR 26-27 · Grupo 9

Martín Luna Aguado y Brayan Guarachi Apaza · Nov 10, 2026

Esta guía sustituye a la anterior y acompaña al proyecto `cnvr-trabajo-final-g9.zip`. Está adaptada al enunciado del 5 de noviembre y a las imágenes que ha publicado el profesor.

0. Qué ha cambiado y en qué estado está el proyecto

0.1 Cambios del enunciado y cómo los he aplicado

Novedad del enunciado (5 de noviembre)	Qué hace ahora el proyecto
Imágenes `dgsgiros/nginx-ssh:latest` (web) y `dgsgiros/admin-ssh:latest` (admin)	Son los valores por defecto. Son públicas en Docker Hub: Kubernetes las descarga solo, no hay que cargar nada a mano
Usuarios `webuser` y `adminuser`, con GID 1000 y `sudo`	El SSH se hace con esos usuarios y las claves van en `/home/<usuario>/.ssh`
Dos pares de claves independientes (paso 5)	Par I: admin → web. Par II: host → admin. Confirma que el texto antiguo tenía las claves al revés
Un Secret por cada clave del admin y un único Secret para todos los web	Tres Secrets: `ssh-admin-private-key`, `ssh-admin-authorized-keys`, `ssh-web-authorized-keys`
Montar el Secret como volumen, con `defaultMode` y `mode`	Las claves se montan como ficheros con modo 0440. Ya no hay initContainers que copien claves
`securityContext.fsGroup: 1000`	Puesto en el Pod admin y en los Pods web
Crear los Secrets con Terraform, no con kubectl	`kubernetes_secret_v1` con la función `file()`
La pública del Par I sirve también para los LXC	VNX la copia con `filetree` en `bbdd` y `logs`

Además he añadido mejoras que suman para la nota:

- Un script que inspecciona las imágenes y el clúster antes de desplegar (`02-inspeccionar-imagenes.sh`).
- Un script de pruebas que marca cada comprobación con [OK] o [FALLO], incluidas las pruebas negativas (`06-pruebas.sh`).
- Un segundo `terraform apply` no muestra cambios (el proveedor ignora las anotaciones que añaden Calico y Multus).
- Si cambia la plantilla HTML, el Deployment recrea los Pods solo.
- Los Pods web no reciben tráfico hasta que Nginx responde (readiness probe).

0.2 Qué está comprobado y qué no

No he podido probar el despliegue. Desde aquí no tengo acceso al laboratorio ni a Docker Hub, así que no he podido ejecutar Terraform ni abrir las imágenes. Esto es lo que sí he comprobado:

- La sintaxis de todos los ficheros `.tf` , del XML de VNX y de los scripts.
- Que todos los bloques y atributos de Terraform existen en el proveedor `hashicorp/kubernetes` 2.23.0, mirando su código fuente.
- Que las variables y los recursos referenciados están definidos.
- Que los manifiestos de instalación de Whereabouts existen en su repositorio.

Quedan tres cosas que solo se pueden confirmar en el laboratorio, y la guía os dice cómo en el Paso 3:

1. **El interior de las imágenes**: directorio raíz de Nginx y configuración de sshd.
2. **La red VlanNet**: nombre de la interfaz en los workers, número de VLAN y si están Multus y Whereabouts.
3. **El XML de VNX**: hay que compararlo con `k8s_lab-vms-vlan.xml` de la práctica 1.3.

Si me pasáis el enunciado de la práctica 1.3 y el fichero `k8s_lab-vms-vlan.xml` , dejo los puntos 2 y 3 exactos.

1. Datos del grupo 9 y direcciones

Dato	Valor
Grupo	9 (Martín Luna Aguado y Brayan Guarachi Apaza)
Ruta del Ingress	`/servers-G9`
Imagen y usuario web	`dgsgiros/nginx-ssh:latest` , usuario `webuser`
Imagen y usuario admin	`dgsgiros/admin-ssh:latest` , usuario `adminuser`

Dato	Valor
GID de ambos usuarios	1000 (se usa como `fsGroup`)

Red	Elemento	Dirección
user_LAN 10.10.20.0/24	Host (IP autorizada para el SSH al admin)	10.10.20.2
user_LAN	Router r1	10.10.20.1
k8s_LAN 10.10.10.0/24	k8s-master	10.10.10.10
k8s_LAN	k8s-worker1 (aquí va el Pod admin)	10.10.10.11
k8s_LAN	k8s-worker2	10.10.10.12
k8s_LAN	terraform	10.10.10.13
VLAN 100, 10.1.100.0/24	Pod admin (IP fija)	10.1.100.10
VLAN 100	Pods web (reparto automático)	10.1.100.100 a 10.1.100.150
VLAN 100	LXC bbdd (MariaDB)	10.1.100.21
VLAN 100	LXC logs (rsyslog)	10.1.100.22

Servicio	Puertos
`web-svc` (ClusterIP)	8080 del Service → 80 de los Pods
`admin-svc` (NodePort)	2222 del Service → 22 del Pod; desde fuera, puerto 30022 de worker1
MariaDB	3306 en bbdd
rsyslog	514 UDP y TCP en logs

Par de claves	Fichero privado	Dónde está la privada	Dónde se autoriza la pública
Par I (admin → web y LXC)	`shared/keys/admin_web_rsa`	Pod admin, vía Secret	Pods web (Secret) y LXC (filetree)

Par de claves	Fichero privado	Dónde está la privada	Dónde se autoriza la pública
Par II (host → admin)	`shared/keys/host_admin_rsa`	Solo en el host	Pod admin, vía Secret

La VLAN 100 y la subred 10.1.100.0/24 las he elegido yo. Si la práctica 1.3 usa otro número de VLAN, cambiadlo en los tres sitios donde aparece: `terraform.tfvars`, `config.sh` y el XML de VNX.

2. Conceptos que hay que entender y explicar en la memoria

2.1 Terraform

En los ficheros `.tf` se describe el estado que se quiere. Terraform lo compara con lo que hay en el clúster y crea, cambia o borra lo necesario. Carga todos los `.tf` de la carpeta como si fueran uno solo; repartirlos en varios es solo por orden.

- `terraform init` descarga el proveedor. `validate` revisa la sintaxis. `plan` enseña lo que va a hacer. `apply` lo hace. `destroy` lo borra.
- **Dependencias implícitas**: si un recurso usa un atributo de otro, Terraform crea antes el segundo. Por ejemplo, el Deployment usa el nombre del Secret y el de la red de Multus, así que ambos se crean primero.
- **Estado**: Terraform guarda en `terraform.tfstate` lo que ha creado. Ese fichero contiene la clave privada del Par I en claro, por eso no se sube a git.

2.2 Multus, VLAN y reparto de IP

- Un Pod tiene por defecto una sola interfaz, `eth0`, que gestiona Calico. **Multus** permite añadir más.
- Cada red extra se describe en un recurso **NetworkAttachmentDefinition** (NAD). El Pod la pide con la anotación `k8s.v1.cni.cncf.io/networks`. La interfaz nueva se llama `net1`.
- En cada worker se crea una subinterfaz VLAN, `eth2.100`. Todo lo que sale por ella lleva la etiqueta 802.1Q con el número 100.
- El plugin **macvlan** crea dentro del Pod una interfaz hija de `eth2.100`, con MAC propia. El Pod queda así conectado a la VLAN 100 de VlanNet, la misma en la que VNX conecta los LXC.
- **Quién da la IP en net1** (IPAM):

  - Pods web: **whereabouts**. Reparte direcciones de un rango sin repetirlas en todo el clúster. El plugin `host-local` lleva una cuenta por nodo, y dos Pods en nodos distintos podrían recibir la misma IP.
  - Pod admin: **static**, con la IP fija 10.1.100.10.

2.3 Secrets como ficheros, permisos y fsGroup

Esta es la parte nueva del enunciado y la que más preguntas puede traer.

- Un Secret montado como volumen aparece dentro del Pod como ficheros. Con `defaultMode` y `mode` se fijan sus permisos.
- Los ficheros pertenecen a `root`. Con **`fsGroup: 1000`**, Kubernetes les pone además el grupo 1000, que es el de `webuser` y `adminuser`.
- Con modo **0440** el fichero queda `r--r-----` de `root:1000`. El usuario puede leerlo por pertenecer al grupo, y nadie puede modificarlo.
- **Clave privada (cliente ssh)**: ssh se niega a usar una clave privada con permisos abiertos, pero solo cuando el fichero es del propio usuario. Aquí el dueño es root, así que `adminuser` puede usarla con 0440. Por la misma razón, ejecutar `ssh` como root dentro del Pod falla con el aviso de clave desprotegida: hay que entrar como `adminuser`.
- **authorized_keys (servidor sshd)**: sshd exige que el fichero y sus directorios sean de root o del usuario y que nadie más pueda escribir en ellos (opción StrictModes).
- **Por qué `sub_path`**: se monta solo el fichero dentro de `~/.ssh`. Si se montara el volumen entero sobre `~/.ssh`, el directorio quedaría con escritura para todos y sshd rechazaría la clave.
- El fichero `~/.ssh/config` del admin no es secreto, por eso va en un ConfigMap.

2.4 Página personalizada: initContainer y Downward API

- Un **initContainer** se ejecuta y termina antes de que arranque el contenedor principal.
- La **Downward API** entrega al contenedor datos del propio Pod como variables de entorno: aquí el nombre del Pod, el nodo y la IP.
- El initContainer lee la plantilla del ConfigMap (volumen de solo lectura), cambia `__POD_NAME__` con `sed` y escribe el resultado en un volumen `emptyDir`. Nginx sirve ese mismo volumen.
- **Por qué la opción B y no un sidecar**: el nombre del Pod no cambia durante su vida, basta generarlo una vez. Un sidecar seguiría consumiendo recursos sin hacer nada.

2.5 Services e Ingress

- **ClusterIP**: IP interna estable que reparte entre los Pods con la etiqueta `app=web`. Solo se ve dentro del clúster.
- **NodePort**: abre un puerto en los nodos (30022) y lo lleva al Pod admin.
- **Ingress**: regla HTTP que aplica el Ingress Controller (un Nginx dentro del clúster). Envía `/servers-G9` a `web-svc:8080`.
- La anotación `rewrite-target: /` cambia la ruta a `/` antes de enviarla al servidor web. Sin ella Nginx buscaría un fichero llamado `servers-G9` y devolvería 404.

2.6 NetworkPolicy

- Sin políticas, todo está permitido. Cuando una política selecciona un Pod, en ese sentido solo pasa lo que digan las reglas.
- Son **aditivas**: lo permitido es la suma de todas las políticas.
- Son **stateful**: las respuestas de una conexión permitida vuelven solas.
- Los puertos de las reglas son los del Pod (80 y 22), no los del Service (8080 y 2222).
- **Solo actúan sobre `eth0`**, la interfaz de Calico. El tráfico por `net1` no pasa por Calico. Por eso el admin llega por SSH a los web y a los LXC aunque esté activa la denegación total.
- Calico siempre deja pasar el tráfico del propio nodo hacia sus Pods (lo necesita kubelet). Por eso las pruebas negativas no se hacen desde el nodo donde está el Pod.

2.7 externalTrafficPolicy: Local y la IP de origen

- Con el valor por defecto (`Cluster`), el nodo puede cambiar la IP de origen por la suya (SNAT) al reenviar al Pod. El Pod no vería la IP del host y la regla `ipBlock` no serviría.
- Con **`Local`**, el nodo solo entrega a Pods que estén en él y no cambia la IP de origen.
- Por eso el Pod admin se fija en `k8s-worker1` con `nodeSelector`, y el SSH se dirige a la IP de ese nodo.

Recorrido completo: host 10.10.20.2 → r1 → 10.10.10.11:30022 → el nodo cambia solo el destino a IP_Pod:22 → Calico comprueba origen 10.10.20.2 y puerto 22 → permitido.

3. Estructura del proyecto

```
cnvr-trabajo-final/
├── README.md                    resumen del proyecto (para la entrega)
├── shared/                      proyecto Terraform; se copia a /shared de la
máquina terraform
|   ├── k8s-provider.tf          proveedor kubernetes 2.23.0
```

```
|   ├── variables.tf              declaración de todas las variables
|   ├── terraform.tfvars          valores del grupo 9
|   ├── locals.tf                 etiquetas, ruta /servers-G9, rutas de .ssh
|   ├── secrets.tf                3 Secrets con las claves SSH
|   ├── multus.tf                 2 NetworkAttachmentDefinition
(kubernetes_manifest)
|   ├── configmap.tf              plantilla HTML y configuración del cliente
SSH del admin
|   ├── html/index.html.tpl       plantilla con el marcador __POD_NAME__
|   ├── web-deployment.tf         Deployment web: initContainer, volúmenes,
fsGroup
|   ├── admin-pod.tf              Pod admin: nodeSelector, fsGroup, claves
|   ├── services.tf               Service ClusterIP (web) y NodePort (admin)
|   ├── ingress.tf                Ingress /servers-G9
|   ├── network-policies.tf       3 NetworkPolicy + 1 temporal opcional
|   ├── outputs.tf                nombres de Pods, IP de la VLAN, comando SSH
|   └── keys/                     aquí se generan las claves (no se suben a
git)
├── vnx/
|   ├── cnvr-lxc-vlan.xml         escenario VNX: LXC bbdd y logs en la VLAN
|   └── conf/
|       ├── ssh/                  authorized_keys (pública del Par I)
|       ├── bbdd/                 60-cnvr.cnf e init.sql
|       └── logs/                 10-remoto.conf (rsyslog)
└── scripts/
    ├── config.sh                   parámetros comunes
    ├── 00-generar-claves.sh        host
    ├── 01-arranque-escenario.sh    host
    ├── 02-inspeccionar-imagenes.sh host (solo consulta)
    ├── 03-vlan-workers.sh          host
    ├── 04-arranque-lxc.sh          host
    ├── 05-terraform-deploy.sh      máquina terraform
    ├── 06-pruebas.sh               host
    ├── 07-cliente-bd.sh            host (opcional)
    ├── 99-borrar.sh                host
    └── desplegar-todo.sh           host (todos los pasos seguidos)
```

Hay dos ficheros con valores que deben coincidir entre sí: `shared/terraform.tfvars` (lo lee Terraform) y `scripts/config.sh` (lo leen los scripts).

Recordad: `/mnt/tmp` se borra al cerrar la sesión del laboratorio. Trabajad siempre en vuestra carpeta personal. El script 01 copia el proyecto a la carpeta compartida y el script 99 guarda una copia de vuelta.

4. Paso 1: preparar el proyecto y generar las claves (host)

```
cd ~
unzip cnvr-trabajo-final-g9.zip
cd cnvr-trabajo-final
chmod +x scripts/*.sh
./scripts/00-generar-claves.sh
```

El script crea los dos pares de claves RSA de 4096 bits, sin frase de paso, en `shared/keys/`:

- `admin_web_rsa` y `admin_web_rsa.pub`: Par I, para que el admin entre en los servidores web y en los LXC.
- `host_admin_rsa` y `host_admin_rsa.pub`: Par II, para que vosotros entréis desde el host al admin.

También copia `admin_web_rsa.pub` a `vnx/conf/ssh/authorized_keys`, que es lo que VNX meterá en los LXC.

Comprobad que existen los cuatro ficheros:

```
ls -l shared/keys/
```

Dos avisos:

- Si volvéis a generar las claves después de desplegar, hay que repetir el despliegue de Terraform y de los LXC. El script no las regenera si ya existen.
- Las claves no se suben a git (están en `.gitignore`). El compañero que clone el repositorio debe ejecutar el script o recibir las claves por otro medio.

Opcional, para no escribir contraseñas. Los scripts entran por SSH como root en los workers y en la máquina terraform, y cada vez piden la contraseña (`xxxx` en los escenarios VNX de la asignatura). Para evitarlo, cuando el escenario esté arrancado:

```
[ -f ~/.ssh/id_rsa ] || ssh-keygen -t rsa -N "" -f ~/.ssh/id_rsa
for n in k8s-master k8s-worker1 k8s-worker2 terraform; do ssh-copy-id
root@$n; done
```

5. Paso 2: arrancar Kubernetes y la máquina terraform (host)

```
./scripts/01-arranque-escenario.sh
```

Hace lo mismo que el apartado 3 de la práctica 1.4:

1. Descarga y arranca el clúster con Calico (`get-k8s-calico` y `start-k8s-calico`).

2. Crea la máquina `terraform` e instala Terraform en ella.
3. Deja las credenciales del clúster en el host y en la máquina terraform (`get-master-kubeconf` y `set-terraform-kubeconf`).
4. Copia la carpeta `shared/` a `/mnt/tmp/k8s_lab-calico-v02/shared`, que la máquina terraform ve como `/shared`.

Comprobad antes de seguir:

```
kubectl get nodes                          # los 3 nodos en Ready
kubectl get pods -n ingress-nginx          # el controlador en Running
kubectl get pods -n kube-system | grep calico
ssh root@terraform terraform --version
ssh root@terraform ls /shared              # deben verse los .tf y keys/
```

Si Terraform no está instalado, repetid `sudo vnx -f k8s_lab-terraform.xml -x install-terraform` desde `/mnt/tmp/k8s_lab-calico-v02`.

Cada vez que cambiéis un fichero de `shared/` en vuestra carpeta personal, hay que volver a copiarlo:

```
cp -r shared/. /mnt/tmp/k8s_lab-calico-v02/shared/
```

6. Paso 3: comprobaciones antes de desplegar (host)

Este paso evita casi todos los fallos posteriores. No modifica nada.

```
./scripts/02-inspeccionar-imagenes.sh | tee inspeccion.txt
```

El script arranca un Pod temporal con cada imagen y muestra su configuración. Después muestra los datos del clúster. Revisad la salida con esta tabla y corregid los valores en `shared/terraform.tfvars` y `scripts/config.sh`:

Qué muestra el script	Qué debe salir	Si sale otra cosa
`id webuser` e `id adminuser`	`gid=1000`	Poned ese número en `ssh_gid`
Home de cada usuario	`/home/webuser` y `/home/adminuser`, sin escritura para grupo ni otros	Si el home es otro, avisadme: hay que cambiar `locals.tf`
Directorio `.ssh`	Puede existir o no; las dos cosas valen	Nada
`sshd_config`	`PubkeyAuthentication` no está en `no`; `AuthorizedKeysFile` no está o apunta a `.ssh/authorized_keys`	Si apunta a otra ruta, hay que cambiar el `mount_path` de `authorized_keys`

Qué muestra el script	Qué debe salir	Si sale otra cosa
Puertos a la escucha	El 22 en las dos imágenes y el 80 en la web	Si falta el 22, la imagen no arranca sshd sola: mirad el apartado de problemas
Directorio raíz de Nginx	Una ruta, normalmente `/usr/share/nginx/html` o `/var/www/html`	Poned esa ruta en `web_root`
Herramientas `mysql` o `mariadb`	Probablemente NO	Usad `07-cliente-bd.sh` para la prueba completa de la base de datos
Interfaces de k8s-worker1	Una interfaz sin IP de k8s_LAN (10.10.10.x) ni de gestión: es la de VlanNet	Poned su nombre en `vlan_parent_if` y en `VLAN_PARENT_IF`
Multus	Pods `multus` en Running y la CRD `network-attachment-definitions`	Instaladlo como en la práctica 1.3
Whereabouts	Puede no estar	El script 03 lo instala
Etiquetas del Ingress Controller	`app.kubernetes.io/name=ingress-nginx` y `app.kubernetes.io/component=controller`	Copiad las reales en `ingress_controller_labels`
IP del Pod del Ingress Controller	Una IP de la red de Pods, distinta de la del nodo	Si es la IP del nodo, mirad el apartado de problemas
Service del Ingress Controller	Tipo NodePort (apuntad el puerto asociado al 80) o LoadBalancer (apuntad la IP externa)	Es la dirección con la que probaréis la web
IngressClass	`nginx`	Poned el nombre real en `ingress_class`

Dos comprobaciones más, a mano:

1. **Número de VLAN.** Abrid `/mnt/tmp/k8s_lab-calico-v02/k8s_lab-vms-vlan.xml` y buscad el bloque `<vlan>` con su `<tag id="...">`. Podéis usar cualquier número, pero tiene que ser el mismo en `vlan_id`, en `VLAN_ID` y en `vnx/cnvr-lxc-vlan.xml`.
2. **IP del host.** Ejecutad `ip -br addr` en el host y confirmad que tiene la 10.10.20.2. Es la IP que autoriza la NetworkPolicy del admin.

Cuando terminéis, volved a copiar el proyecto a la carpeta compartida (comando del Paso 2).

7. Paso 4: VLAN en los workers y Multus (host)

Corresponde al apartado 2, pasos 2 y 3 del enunciado.

```
./scripts/03-vlan-workers.sh
```

En cada worker ejecuta lo que hicisteis a mano en la práctica 1.3:

```
ip link set eth2 up
ip link add link eth2 name eth2.100 type vlan id 100
ip link set eth2.100 up
```

Después comprueba que Multus está instalado e instala Whereabouts si falta.

Comprobad:

```
ssh root@k8s-worker1 ip -d link show eth2.100     # debe decir: vlan protocol
802.1Q id 100
ssh root@k8s-worker2 ip -d link show eth2.100
kubectl get crd | grep -E 'network-attachment|whereabouts'
kubectl get pods -A | grep -Ei 'multus|whereabouts'   # todos en Running
```

Tres notas:

- La subinterfaz no sobrevive a un reinicio del worker. Si reiniciáis el escenario, repetid este script.
- La subinterfaz no lleva IP. Solo los Pods y los LXC tienen dirección en la VLAN.
- Si en la práctica 1.3 la red de Multus se definió de otra forma (otro plugin u otra interfaz), decidme cómo era y adapto `multus.tf`. La idea no cambia: una red de Multus cuya interfaz padre es la subinterfaz VLAN.

8. Paso 5: contenedores LXC con VNX (host)

Corresponde al apartado 2, paso 6 del enunciado.

8.1 Antes de arrancar: comparar con el XML de la práctica 1.3

El enunciado dice que se parta de `k8s_lab-vms-vlan.xml`. Mi fichero `vnx/cnvr-lxc-vlan.xml` sigue esa idea, pero no he visto el original. Abrid los dos y comparad estas tres cosas. Si difieren, copiad en el mío lo que ponga el del laboratorio:

Elemento	En mi XML	Qué hacer
Red compartida	`<net name="VlanNet" mode="openvswitch"/>`	El nombre y el `mode` deben ser idénticos a los del laboratorio

Elemento	En mi XML	Qué hacer
Sistema de ficheros de los LXC	`/usr/share/vnx/filesystems/rootfs_lxc_ubuntu64`	Usad la misma ruta que el laboratorio
VLAN en la interfaz	Bloque `<vlan>` con `<tag id="100"/>` dentro del `<if>`	Misma sintaxis y mismo número que en los workers

Mirad también el atributo `offset` de `<automac>`. El mío es 50. Si el escenario del laboratorio usa el mismo valor, cambiad el mío para que no se repitan direcciones MAC.

8.2 Arrancar y configurar

```
./scripts/04-arranque-lxc.sh
```

El script crea el escenario y lanza la secuencia `configurar`. Tarda unos minutos porque instala paquetes.

Qué define el XML:

- Dos contenedores: **bbdd** (10.1.100.21) y **logs** (10.1.100.22).
- Cada uno tiene dos interfaces. La primera está en VlanNet, etiquetada en la VLAN 100. La segunda está en `virbr0` con DHCP y solo sirve para que `apt-get` tenga Internet.
- Bloques **filetree**: copian del host al contenedor, en `/root/conf/`, las carpetas `conf/ssh`, `conf/bbdd` y `conf/logs`.
- Bloques **exec**: ejecutan comandos dentro del contenedor, en este orden:
  1. Instalan `mariadb-server` (o `rsyslog`) y `openssh-server`.
  2. Copian `authorized_keys` a `/root/.ssh` con permisos 700 y 600, y desactivan el acceso con contraseña.
  3. En bbdd: copian `60-cnvr.cnf`, que pone `bind-address = 0.0.0.0`. Por defecto MariaDB solo escucha en localhost; es el fallo que tuvo el grupo del año pasado. Después ejecutan `init.sql`.
  4. En logs: copian `10-remoto.conf`, que hace que rsyslog escuche en el puerto 514 (UDP y TCP) y guarde los mensajes de cada origen en `/var/log/remoto/<origen>.log`.

Qué crea `init.sql`:

- La base de datos `cnvr`.

- El usuario `cnvr` con contraseña `cnvr2627`, que solo puede conectarse desde la VLAN (`10.1.100.%`).
- La tabla `accesos`, que usan las pruebas para dejar constancia de quién se conecta.

8.3 Comprobar

```
sudo lxc-attach -n bbdd -- ip -4 -br addr              # 10.1.100.21
sudo lxc-attach -n bbdd -- ss -ltn | grep -E ':(22|3306) '   # 0.0.0.0:3306 y
:22
sudo lxc-attach -n bbdd -- mysql -e 'SHOW DATABASES; SELECT user,host FROM
mysql.user;'
sudo lxc-attach -n logs -- ss -ltnu | grep ':514 '
sudo lxc-attach -n logs -- ping -c 2 10.1.100.21       # los dos LXC se ven
por la VLAN
```

Si `apt-get` falla dentro de los contenedores, es que no tienen Internet por `virbr0`. Comprobadlo con `sudo lxc-attach -n bbdd -- ping -c 2 8.8.8.8`.

9. Paso 6: desplegar con Terraform

Hay dos formas. Usad la primera para aprender y para la memoria, y la segunda para la demostración final.

- **Por partes**, con `-target`, como en la práctica 1.4. Se ve el efecto de cada recurso y se pueden capturar el antes y el después de las NetworkPolicy.
- **Todo seguido**: `ssh root@terraform 'bash /shared/scripts/05-terraform-deploy.sh'`.

Para ir por partes, entrad en la máquina terraform. Los comandos `kubectl` podéis lanzarlos desde otra terminal del host.

```
ssh root@terraform
cd /shared
terraform init
terraform fmt          # deja el formato estándar
terraform validate     # debe decir: Success! The configuration is valid.
```

Si `validate` da un error, copiadme el mensaje completo: indica el fichero y la línea.

9.1 Secrets y ConfigMaps

```
terraform apply \
  -target=kubernetes_secret_v1.ssh_admin_privada \
  -target=kubernetes_secret_v1.ssh_admin_autorizadas \
  -target=kubernetes_secret_v1.ssh_web_autorizadas \
  -target=kubernetes_config_map_v1.plantilla_web \
  -target=kubernetes_config_map_v1.ssh_config_admin
```

Comprobad:

```
kubectl get secrets,configmaps
kubectl describe secret ssh-admin-private-key      # muestra el tamaño de
id_rsa, no su contenido
kubectl get secret ssh-web-authorized-keys -o
jsonpath='{.data.authorized_keys}' | base64 -d
kubectl get configmap plantilla-web -o yaml        # se ve el marcador
__POD_NAME__ y "Grupo 9"
```

Qué explicar:

- `file("keys/admin_web_rsa")` lee el fichero al aplicar. Es el equivalente en Terraform de `kubectl create secret generic --from-file`.
- En `data` se escribe el texto tal cual y el proveedor lo codifica en base64. Un Secret no está cifrado, solo codificado.
- `templatefile()` rellena el número de grupo en la plantilla. Los marcadores `__POD_NAME__`, `__NODE_NAME__` y `__POD_IP__` no son sintaxis de Terraform y llegan intactos.

9.2 Redes de Multus

```
terraform apply -target=kubernetes_manifest.nad_vlan_web -
target=kubernetes_manifest.nad_vlan_admin
kubectl get network-attachment-definitions
kubectl get net-attach-def vlan-web -o yaml
```

Qué explicar:

- El proveedor no tiene un recurso propio para este tipo, así que se usa `kubernetes_manifest`, que acepta cualquier objeto de Kubernetes escrito en HCL.
- El campo `spec.config` es un texto en JSON; por eso se genera con `jsonencode()`.
- `kubernetes_manifest` consulta el clúster ya en el `plan`. Si Multus no está instalado, falla aquí.

9.3 Deployment de los servidores web

```
terraform apply -target=kubernetes_deployment_v1.web
```

Terraform espera a que los 3 Pods estén listos. Si tarda más de dos minutos, mirad en otra terminal `kubectl get pods` y `kubectl describe pod <pod>`.

Comprobad, cambiando `<pod>` por el nombre de uno de los Pods web:

```
kubectl get pods -l app=web -o wide
kubectl describe pod <pod>                # Init Containers: genera-index,
estado Completed
kubectl logs <pod> -c genera-index        # "index.html generado para
web-..."
kubectl exec <pod> -c web -- ip -4 addr show net1       # una IP
10.1.100.1xx
kubectl exec <pod> -c web -- ls -ln /home/webuser/.ssh/  # authorized_keys: -
r--r----- 0 1000
kubectl exec <pod> -c web -- curl -s localhost | grep web-    # la página con
el nombre del Pod
```

Qué ocurre al arrancar cada Pod:

1. Multus lee la anotación y añade la interfaz `net1`, con una IP que le da Whereabouts.
2. Se ejecuta el initContainer `genera-index`. Recibe el nombre del Pod por la Downward API, lee la plantilla del ConfigMap (solo lectura) y escribe `index.html` en el volumen `emptyDir`.
3. Arranca el contenedor `web`. Nginx sirve el `emptyDir`. La clave pública del Par I está montada desde el Secret en `/home/webuser/.ssh/authorized_keys`.
4. Cuando Nginx responde, el Pod pasa a Ready y el Service empieza a enviarle tráfico.

9.4 Service web e Ingress

```
terraform apply -target=kubernetes_service_v1.web -
target=kubernetes_ingress_v1.web
kubectl get svc web-svc                # ClusterIP, 8080/TCP
kubectl get endpoints web-svc          # las 3 IP de los Pods, puerto 80
kubectl describe ingress web-ingress   # /servers-G9 -> web-svc:8080
```

Probad desde el host. Usad la dirección del Ingress Controller que apuntasteis en el Paso 3:

```
curl http://10.10.10.11:<puerto_del_ingress>/servers-G9
for i in $(seq 1 9); do curl -s http://10.10.10.11:
<puerto_del_ingress>/servers-G9 | grep 'class="pod"'; done
```

Deben aparecer los nombres de los tres Pods. Abrid también la URL en el navegador para la captura de la memoria.

9.5 Pod y Service del admin

```
terraform apply -target=kubernetes_pod_v1.admin -
target=kubernetes_service_v1.admin
kubectl get pod admin -o wide                       # NODE: k8s-worker1
kubectl get svc admin-svc                           # NodePort,
2222:30022/TCP
kubectl get svc admin-svc -o jsonpath='{.spec.externalTrafficPolicy}'    #
Local
kubectl exec admin -- ls -ln /home/adminuser/.ssh/  # id_rsa, authorized_keys
y config, de 0:1000
kubectl exec admin -- ip -4 addr show net1          # 10.1.100.10
```

Entrad desde el host (en vuestra carpeta del proyecto):

```
ssh -i shared/keys/host_admin_rsa -p 30022 adminuser@10.10.10.11
```

Ya dentro del admin:

```
echo $SSH_CLIENT           # debe empezar por 10.10.20.2: el Pod ve la IP
real del host
ping -c 2 10.1.100.21      # bbdd por la VLAN
ssh bbdd hostname          # entra como root en el LXC, sin contraseña
ssh logs hostname
ssh 10.1.100.1xx hostname  # un servidor web (IP de net1), entra como webuser
```

9.6 NetworkPolicy: probar antes y después

Antes de aplicarlas, preparad un Pod de pruebas en otro namespace y comprobad que todo está abierto. Guardad captura.

```
kubectl create namespace pruebas-np
kubectl -n pruebas-np run tester --image=busybox:1.36 --restart=Never --
command -- sleep 3600
kubectl get pods -l app=web -o wide       # apuntad la IP (columna IP) de un
Pod web
kubectl -n pruebas-np exec tester -- wget -T 5 -qO- http://<IP_pod_web>
# devuelve la página
ssh root@terraform "timeout 5 bash -c '</dev/tcp/10.10.10.11/30022' && echo
ABIERTO || echo BLOQUEADO"   # ABIERTO
```

Aplicad las tres políticas:

```
terraform apply \
  -target=kubernetes_network_policy_v1.default_deny_all \
  -target=kubernetes_network_policy_v1.web_desde_ingress \
  -target=kubernetes_network_policy_v1.admin_ssh_desde_host
kubectl get networkpolicy
kubectl describe networkpolicy web-allow-from-ingress-nginx
kubectl describe networkpolicy admin-allow-ssh-from-host
```

Después, repetid las dos pruebas: el `wget` debe agotar el tiempo y la segunda debe decir BLOQUEADO. En cambio, deben seguir funcionando el `curl` al Ingress y el SSH desde el host. Borrad el Pod de pruebas con `kubectl delete namespace pruebas-np`.

Qué hace cada política:

1. **default-deny-all**. `pod_selector {}` selecciona todos los Pods del namespace. Declara los tipos Ingress y Egress sin ninguna regla, así que no se permite nada.
2. **web-allow-from-ingress-nginx**. Selecciona los Pods `app=web` y permite la entrada al puerto 80. El origen lleva `namespace_selector` y `pod_selector` en el mismo bloque `from`, lo que significa que deben cumplirse los dos. En dos bloques `from` separados bastaría con cumplir uno. El Ingress Controller se conecta directamente a las IP de los Pods, no a la IP del Service; por eso el puerto de la regla es el 80.
3. **admin-allow-ssh-from-host**. Selecciona el Pod `app=admin` y permite la entrada al puerto 22 solo desde `10.10.20.2/32`, con `ip_block`.

Las respuestas salen aunque la salida esté denegada, porque las políticas son stateful.

9.7 Aplicar el resto y revisar

```
terraform apply                                  # crea lo que falte y muestra
los outputs
terraform apply -refresh-only -auto-approve    # vuelve a leer las IP de la
VLAN
terraform output
terraform state list                             # todos los recursos
gestionados
terraform plan                                   # debe decir: No changes
```

El output `web_pods_ip_vlan` da la IP de la VLAN de cada Pod web. La lee de una anotación que Multus escribe en el Pod.

Probad también el escalado, que es lo que da sentido a "aplicación escalable":

```
terraform apply -var="num_replicas=5"
kubectl get pods -l app=web -o wide       # 5 Pods, cada uno con su página y
su IP en la VLAN
terraform apply                           # vuelve a 3
```

10. Paso 7: pruebas

Corresponde al apartado 2, paso 7, y sirve para demostrar los apartados 3, 4 y 5.

```
./scripts/06-pruebas.sh | tee pruebas.txt
```

El script marca cada comprobación con [OK] o [FALLO] y termina con un resumen. Haced una captura de cada bloque para la memoria. Esta tabla dice qué demuestra cada prueba y cómo repetirla a mano:

Prueba	Qué demuestra	Cómo hacerla a mano	Resultado esperado
P1	Todos los recursos existen	`kubectl get deploy,pods,svc,ingress,networkpolicy,net-attach-def -o wide`	3 web y admin en Running; admin en k8s-worker1
P2	Interfaces de Multus	`kubectl exec <pod> -c web -- ip -4 addr show net1`	Una IP 10.1.100.x en cada Pod

Prueba	Qué demuestra	Cómo hacerla a mano	Resultado esperado
P3	Página personalizada, Service e Ingress	`curl` repetido a la URL del Ingress con `/servers-G9`	Código 200 y los nombres de los 3 Pods; otra ruta da 404
P4	SSH host → admin con el Par II y conservación de la IP origen	`ssh -i shared/keys/host_admin_rsa -p 30022 adminuser@10.10.10.11` y dentro `echo $SSH_CLIENT`	Entra sin contraseña; el origen es 10.10.20.2
P5	Solo el host puede entrar al admin	Mismo SSH desde la máquina terraform, y desde el host contra 10.10.10.12	No conecta en ninguno de los dos casos
P6	Denegación por defecto y acceso web solo desde el Ingress Controller	Desde un Pod de otro namespace: `wget` a la IP de un Pod web y a la IP de `web-svc`; desde un Pod web, `curl` a otro Pod web	Todo bloqueado, salvo el acceso a través del Ingress
P7	Conectividad por la VLAN	Desde el admin: `ping` a cada web (IP de net1), a 10.1.100.21 y a 10.1.100.22	Responden todos
P8	SSH admin → web y LXC con el Par I	Desde el admin: `ssh <IP_net1_web>`, `ssh bbdd`, `ssh logs`	Entra sin contraseña como webuser y como root
P9	El admin y los web llegan a la base de datos	Desde el admin y desde un web: `nmap -Pn -p 3306 10.1.100.21`; desde el admin: `ssh bbdd mysql -e 'SHOW DATABASES'`	Puerto abierto; se listan las bases de datos
P10	Servidor de logs	Desde el admin: `logger -n 10.1.100.22 -P 514 -d "prueba"` y luego `ssh logs 'tail /var/log/remoto/*.log'`	El mensaje aparece en el servidor de logs

Pruebas que se hacen aparte:

- **P11, la VLAN no depende de las políticas.** P7 y P8 funcionan con la denegación total activa. Explicad por qué (apartado 2.6).
- **P12, base de datos con cliente SQL.** Si las imágenes no traen cliente, `./scripts/07-cliente-bd.sh` lo instala y después P9 hace una inserción y una consulta desde el admin y desde un web. El script crea con Terraform una política temporal que permite la salida, instala `mariadb-client` con `apt` y elimina la política. Lo instalado se pierde si el Pod se recrea.
- **P13, Terraform.** `terraform plan` sin cambios, escalado a 5 réplicas y `terraform destroy` seguido de `terraform apply` para demostrar que el despliegue es repetible.

Para repetir P12 a mano desde el admin, una vez instalado el cliente:

```
mysql -h 10.1.100.21 -u cnvr -pcnvr2627 cnvr -e "INSERT INTO
accesos(origen,mensaje) VALUES (USER(), 'prueba'); SELECT * FROM accesos;"
```

La columna `origen` guarda el usuario y la IP desde la que se conectó el cliente, así queda a la vista qué máquina hizo cada inserción.

Tres cuidados al hacer pruebas negativas:

- No las hagáis desde el nodo donde está el Pod: Calico deja pasar siempre el tráfico del nodo a sus propios Pods.
- No uséis un Pod del namespace `default` como origen: su propia salida está denegada y la prueba no diría nada sobre el destino. Por eso el script crea el Pod de pruebas en otro namespace.
- Dentro del admin, usad `ssh` como `adminuser`. Con `kubectl exec` se entra como root y ssh rechaza la clave (apartado 2.3).

11. Problemas típicos y cómo resolverlos

Síntoma	Causa probable	Solución
`terraform plan` dice "no matches for kind NetworkAttachmentDefinition"	Multus no está instalado	Instaladlo como en la práctica 1.3
`terraform apply` falla con "no such file" en `keys/...`	No se generaron las claves o no se copiaron a `/shared`	Script 00 y volver a copiar `shared/`

Síntoma	Causa probable	Solución
Pod en ContainerCreating y `describe` nombra whereabouts	Whereabouts no está instalado o aún arranca	`kubectl -n kube-system get ds whereabouts`; repetid el script 03
Pod en ContainerCreating y `describe` dice "Link not found"	No existe `eth2.100` en ese worker o la interfaz se llama de otra forma	Revisad `vlan_parent_if` y repetid el script 03
Pod en ImagePullBackOff	El worker no llega a Docker Hub o se alcanzó su límite de descargas	`kubectl describe pod`; esperad unos minutos o reintentad
Pods web en Running pero 0/1 Ready	Nginx no responde en el puerto 80	`kubectl logs <pod> -c web` y revisad la salida del script 02
La página es la de bienvenida de Nginx	`web_root` no es el directorio que usa la imagen	Corregid `web_root` con el dato del script 02 y aplicad
`curl` al Ingress da 404	Ruta mal escrita (es `/servers-G9`, con G mayúscula) o IngressClass distinta	`kubectl describe ingress web-ingress` y `kubectl get ingressclass`
`curl` al Ingress da 504 tras aplicar las políticas	Las etiquetas del controlador no coinciden con la política	Copiad las reales en `ingress_controller_labels`
Sigue el 504 con las etiquetas bien	El Pod del controlador usa la red del nodo (su IP es la del nodo)	Con `pod_selector` no se puede seleccionar; consultad al profesor
El host no entra al admin tras aplicar las políticas	El Pod no ve 10.10.20.2 como origen, o el SSH no va a worker1	Quitad la política del admin, entrad y mirad `echo $SSH_CLIENT`; poned esa IP en `host_ip` solo si es la del host
SSH pide contraseña o dice "Permission denied (publickey)"	sshd no acepta el fichero `authorized_keys`	Diagnóstico de abajo

Síntoma	Causa probable	Solución
ssh dice "UNPROTECTED PRIVATE KEY FILE"	Se está ejecutando ssh como root dentro del admin	Entrad como `adminuser`
No hay ping entre admin y LXC	Número de VLAN distinto en workers y VNX, o interfaz equivocada	En el worker: `tcpdump -e -n -i eth2 vlan` y mirad las etiquetas
`mysql` dice "Host is not allowed"	El usuario `cnvr` solo admite conexiones desde 10.1.100.x	Conectaos por la VLAN
`terraform plan` siempre quiere cambiar el Pod admin	Alguna anotación externa no está en `ignore_annotations`	Añadid la anotación que muestre el plan en `k8s-provider.tf`
Cambios en un `.tf` que no se aplican	Editasteis la copia personal y no la de `/shared`	Volved a copiar `shared/`

Diagnóstico cuando SSH no acepta la clave. Arrancad un segundo sshd en modo depuración dentro del Pod web y conectaos a él desde el admin:

```
# Terminal 1 (host): sshd en modo depuración en el puerto 2200 de un Pod web
kubectl exec -it <pod> -c web -- sudo /usr/sbin/sshd -d -p 2200
# Terminal 2 (dentro del admin):
ssh -p 2200 <IP_net1_de_ese_pod>
```

La terminal 1 dice el motivo exacto. Los dos más probables:

- "bad ownership or modes for directory /home/webuser": el directorio personal de la imagen tiene escritura para el grupo.
- "Could not open authorized keys": el usuario no puede leer el fichero; revisad `ssh_gid`.

En cualquiera de los dos casos, pasadme el mensaje y la salida de `kubectl exec <pod> -c web -- ls -lan /home/webuser /home/webuser/.ssh`. Tengo preparada una alternativa que copia la clave con un initContainer y fija los permisos a mano.

Si el puerto 22 no está a la escucha en la salida del script 02, la imagen no arranca sshd por sí sola. Pasadme el fichero `inspeccion.txt` y añado el arranque al contenedor.

12. Memoria, entrega y lista para el 10

12.1 Cada requisito, dónde se cumple y cómo se demuestra

Poned esta tabla en la memoria. Al corrector le permite comprobar de un vistazo que no falta nada.

Requisito del enunciado	Fichero	Prueba
2.1 Imágenes proporcionadas y sus usuarios	`terraform.tfvars`, `web-deployment.tf`, `admin-pod.tf`	P1, P4, P8
2.2 VLAN en los workers	`scripts/03-vlan-workers.sh`	P2, P7
2.3 NetworkAttachmentDefinition con Multus	`multus.tf`	P2
2.4 SSH por la VLAN con clave, sin contraseña	`admin-pod.tf`, `web-deployment.tf`, `configmap.tf`	P4, P8
2.5 Claves en Secrets creados con Terraform, dos pares, fsGroup	`secrets.tf`, `scripts/00-generar-claves.sh`	P4, P8
2.6 LXC con VNX: MariaDB, logs, SSH, exec y filetree	`vnx/cnvr-lxc-vlan.xml`, `vnx/conf/`	P7 a P10
2.7 Verificación de conectividad	`scripts/06-pruebas.sh`	P2, P7, P8, P9
3.1 ConfigMap con la plantilla	`configmap.tf`, `html/index.html.tpl`	P3
3.2 Generación de la página (opción B, initContainer)	`web-deployment.tf`	P3
3.3 Volumen del ConfigMap en solo lectura y volumen compartido	`web-deployment.tf`	P3
4.1 Service ClusterIP 8080 → 80	`services.tf`	P1, P3
4.2 Service NodePort 2222 → 22	`services.tf`	P1, P4
4.3 Ingress `/servers-G9`	`ingress.tf`	P3
5.1 Denegación por defecto	`network-policies.tf`	P6
5.2 Web solo desde el Ingress Controller	`network-policies.tf`	P3, P6
5.3 Admin solo por SSH desde el host, IP origen conservada, nodeSelector	`network-policies.tf`, `services.tf`, `admin-pod.tf`	P4, P5

12.2 Estructura de la memoria

1. **Introducción**: objetivo y enfoque de infraestructura como código.
2. **Arquitectura**: la Figura 1 con vuestras direcciones y las tablas del apartado 1 de esta guía.

3. **Desarrollo**, un subapartado por cada apartado del enunciado. En cada uno: qué se pide, cómo lo habéis resuelto (con el fragmento de código clave) y por qué.
4. **Decisiones de diseño**, que es donde se gana nota:
   - Opción B (initContainer) frente al sidecar.
   - Whereabouts para los web e IP fija para el admin.
   - Montaje de los Secrets con `sub_path`, modo 0440 y `fsGroup`.
   - Configuración del cliente SSH en un ConfigMap y por qué no se guardan las claves de host de Pods efímeros.
   - `externalTrafficPolicy: Local` con `nodeSelector`.
   - Por qué las políticas no afectan a la VLAN.
   - Todo parametrizado con variables y `locals`.
5. **Automatización**: los scripts, su orden y el despliegue con un solo comando.
6. **Pruebas**: P1 a P13, cada una con captura, comando y una frase que explique el resultado. Incluid las negativas y el antes y después de las políticas.
7. **Problemas encontrados y soluciones**. Los que os salgan de verdad en el laboratorio.
8. **Limitaciones y mejoras**: la clave privada queda en el estado de Terraform; el cliente SQL instalado a mano se pierde al recrear el Pod; la subinterfaz VLAN no persiste a reinicios.
9. **Conclusiones**.

12.3 Lista de comprobación antes de entregar

- [ ] `terraform fmt` y `terraform validate` sin errores.
- [ ] `terraform destroy` seguido de `desplegar-todo.sh --sin-k8s` funciona de principio a fin.
- [ ] `06-pruebas.sh` termina con 0 fallos.
- [ ] Un segundo `terraform plan` dice "No changes".
- [ ] La ruta es exactamente `/servers-G9`.
- [ ] Los ficheros entregados son los de `/shared` tras el `terraform fmt`, copiados de vuelta a vuestra carpeta.
- [ ] La entrega no incluye claves privadas, ni `terraform.tfstate`, ni la carpeta `.terraform`.
- [ ] La memoria incluye la tabla de requisitos, las capturas y los nombres de los dos.
- [ ] Habéis mirado en Moodle el formato y el plazo de entrega; el enunciado no los indica.

12.4 Preguntas que os pueden hacer

- **¿Por qué el admin llega a los web por SSH si hay una denegación total?** Porque va por `net1`, que no pasa por Calico.
- **¿Qué pasa si quitáis `externalTrafficPolicy: Local`?** El Pod puede ver como origen la IP de un nodo y la regla `ip_block` deja de coincidir.
- **¿Por qué el puerto de la política web es el 80 y no el 8080?** Porque la política se aplica en el Pod, después de que el Service traduzca el puerto.
- **¿Por qué la clave privada tiene modo 0440 y funciona?** Porque el fichero es de root y ssh solo exige permisos privados cuando es del propio usuario; `adminuser` la lee por el grupo 1000 que pone `fsGroup`.
- **¿Qué pasa si muere un Pod web?** El Deployment crea otro; el initContainer genera su página con el nombre nuevo y Whereabouts le da una IP de la VLAN.
- **¿Un Secret está cifrado?** No, solo codificado en base64. Quien pueda leer Secrets en el namespace puede leer la clave.
- **¿Por qué `kubernetes_manifest` para Multus?** Porque es un tipo de recurso añadido al clúster y el proveedor no tiene un recurso específico para él.

13. Orden de ejecución resumido

```
# En el host, dentro de ~/cnvr-trabajo-final
./scripts/00-generar-claves.sh                # Paso 1
./scripts/01-arranque-escenario.sh            # Paso 2
./scripts/02-inspeccionar-imagenes.sh         # Paso 3: revisar y corregir
terraform.tfvars y config.sh
cp -r shared/. /mnt/tmp/k8s_lab-calico-v02/shared/
./scripts/03-vlan-workers.sh                  # Paso 4
./scripts/04-arranque-lxc.sh                  # Paso 5
ssh root@terraform 'bash /shared/scripts/05-terraform-deploy.sh'    # Paso 6
./scripts/06-pruebas.sh                       # Paso 7
./scripts/07-cliente-bd.sh                    # opcional, y repetir 06
./scripts/99-borrar.sh                        # al terminar: copia de
seguridad y borrado
```

Cuando los valores ya estén confirmados, todo el despliegue se lanza con un solo comando:

```
./scripts/desplegar-todo.sh             # incluye arrancar Kubernetes
./scripts/desplegar-todo.sh --sin-k8s   # si el clúster ya está arrancado
```

Plan para la primera sesión de laboratorio: Pasos 1 a 3 y corregir valores. En esa misma sesión, o en la siguiente, Pasos 4 a 7. Si algo falla, pasadme el mensaje de error y los ficheros `inspeccion.txt` y `pruebas.txt`.